

Cache-Sensitive Memory Layout for Dynamic Binary Trees

RIKU SAIKKONEN* AND ELJAS SOISALON-SOININEN

*Department of Computer Science, School of Science, Aalto University,
PO Box 15400, FI-00076 Aalto, Finland*

**Corresponding author: riku.saikkonen@alumni.aalto.fi*

We use cache-sensitive memory layouts to improve search performance in ordinary binary search trees, without increasing the time complexity of insertion or deletion. Our approach does not require changes to the structure of the nodes or the rebalancing strategy of the tree. Cache-sensitivity is maintained with a memory-layout invariant that keeps a given number of neighboring nodes in the tree stored in the same cache block. We show how to preserve it using worst-case constant-time operations executed when the tree changes. In our extensive experiments, the invariant consistently improved the search performance of large AVL and red-black trees by 26–32%, on synthetic data as well as realistic search tree operations extracted from a database benchmark. We also compare binary search trees with several cache-sensitive B-trees, and non-dynamic approaches for laying out tree nodes in a multi-level memory hierarchy.

Keywords: binary search trees; cache-conscious data structures; experiments

Received 20 November 2014; revised 23 August 2015

Handling editor: Rajeev Raman

1. INTRODUCTION

Almost all current computers have a hierarchical memory system, where a number of hardware caches are placed between the processor and the main memory. Caches are small but fast memories that provide faster access to recently used memory locations. Caching has become an important factor in the practical performance of main-memory data structures. Its relative importance will likely continue to increase [1,2], especially since many applications that previously needed to read data from disk can now fit all of the necessary data in the main memory. In data-intensive main-memory applications, reading from main memory is often a bottleneck similar to disk I/O for external-memory algorithms.

There are two types of cache-centric algorithms. We focus on the *cache-sensitive* (also called *cache-conscious* or *cache-aware*) model, where the implementation knows the specific parameters of the caches that are in use and adapts itself to them. In contrast, *cache-oblivious* algorithms attempt to optimize themselves to an unknown memory hierarchy.

For search trees, the most important cache parameters are the *cache block sizes*, i.e. the granularities at which data are stored in the caches and transferred between levels of the memory hierarchy. The primary goal is to enhance search performance by

reducing the number of separate cache blocks visited on a search path [3–5].

In this article, we show how to preserve a cache-sensitive memory layout in the presence of insertions and deletions to binary search trees. No general results of this kind have been presented thus far: most cache-centric search trees are variants of the B-tree, and the scarce research on the effect of caches on binary trees consists of methods for rearranging all nodes of a fixed tree into a good memory layout. While the adjustable node size makes B-tree-like structures more suitable for cache memories, we feel that it is also useful to improve binary search trees, since they are very commonly used in real-world software. One recent use of binary search trees is in database systems that employ copy-on-write semantics: binary trees are optimal for copying root-to-leaf paths in update operations [6]. This is why, for instance, Bernstein *et al.* [7] used (non-cache-optimized) binary search trees as the main index structure of their Hyder prototype database, despite their mention of poor processor cache performance as a disadvantage.

Constraints that we set upon ourselves were not to increase the time complexity of insertion or deletion (more than a constant factor), and to preserve both the structure of the tree nodes and the rebalancing algorithm intact. This makes our

solution apply to any binary search tree that uses rotations for balancing: the usual AVL and red–black trees as well as many others, for example rank-balanced trees [8] (a recent binary search tree that combines some of the best properties of AVL (Adelson-Velsky and Landis) and red–black trees) and more special-purpose structures like stratified trees [9] (designed to optimize bulk operations). In contrast, most other research on cache-sensitive or cache-oblivious search trees only works with one specific kind of tree or even dictates its own cache-centric rebalancing algorithm for the tree. (We discuss related work in detail below.)

Our solution is based on a simple memory-layout invariant: in its simplest form, we require a non-leaf node to be located on the same cache block as either its parent or its child. We re-establish the invariant in constant time whenever the tree changes in insertion or deletion or when a rotation is performed. Even this simple invariant increases search performance significantly, but we also discuss an extension that keeps a subtree containing at least $\alpha > 2$ connected nodes on the same cache block, where α is a user-selectable parameter. We show that the simple invariant can reduce the number of separate cache blocks traversed by a search by 33%, or up to 50% with $\alpha > 2$.

Another contribution of the present article is in our extensive experimentation, especially (to our knowledge) the first study of cache-sensitive search trees using real-world inputs, taken from the well-known TPC-C database benchmark [10]. We compare our solution to standard AVL and red–black trees, and to a non-dynamic multi-level layout that is not preserved in insertions or deletions. Moreover, we include several B-trees in our experiments: ordinary B^+ -trees with cache-block-sized nodes (which are already quite efficient [3]) and the cache-sensitive B^+ -trees of Rao and Ross [1]. We used synthetic (random) input operations as well as operation sequences from the TPC-C benchmark, which we acquired by instrumenting MySQL to produce a trace of its search-tree operations.

With regard to our approach for preserving a cache-sensitive memory layout in insertions and deletions, the main result of our experiments is that it consistently improved search time by 26–32% in red–black and AVL trees having $\sim 10^7$ keys. This was true on four different computers, and on both random searches and the non-random access pattern of the TPC-C benchmark. It is also consistent with our theoretical analysis (the 33% referred to above).

On random searches, we found the non-dynamic multi-level layout to improve the search time of red–black and AVL trees by 51–56%, as well as improving B^+ -trees with cache-block-sized nodes by 19–34% and cache-sensitive B^+ -trees by 1–12%. Searches extracted from TPC-C exhibited closely similar (slightly smaller) improvements. We consider the 26–32% improvement produced by our dynamic approach to be quite good when compared to the 51–56% of the much more complex and non-dynamic layout.

A disadvantage of our approach is that it increases insertion and deletion times, though only by a constant factor. Here

experimental results varied in the computers we tried, but generally preserving the invariant made random insertions $\sim 50\%$ slower to twice as slow, while deletions were only slightly slower, sometimes even faster (the invariant speeds up searching for the affected node). The TPC-C operations have $\sim 9\%$ insertions, making our dynamic approach considerably less helpful than with only searches: we got at best 9% improvement (compared with 26–30% for only searches). However, our prototype implementation was not optimized for insertion and deletion times.

This article is organized as follows. We define our cache model in Section 2. Section 3 gives the (parameterized) invariant that our dynamic single-level algorithm preserves, analyzes its effect on tree searches and presents the algorithm that preserves this invariant. We describe our experiments in Section 4, giving the results in Section 5. Section 6 discusses related work, Section 7 compares binary search trees with B-trees and Section 8 concludes the article.

2. CACHE MODEL

The memory architecture of a typical modern computer consists of several levels of fast but relatively small cache memories before the slower and larger main memory. Data are transferred between consecutive levels of the memory hierarchy in *cache blocks*, which have a fixed size at every level.

Figure 1 gives an example of a hypothetical memory hierarchy with two levels of cache memories using 64- and 256-byte cache blocks. When a word of 4 bytes is read from memory and is not present in either of the caches, the 256-byte cache block containing the word is first copied from the main memory into the larger cache. Then the 64-byte cache block containing the word is copied from the larger cache to the smaller one, and finally the 4 bytes are read into a CPU register from the smaller cache.

In addition to the data caches, the mapping of virtual addresses to physical addresses used by multi-tasking operating systems employs another kind of cache: the *Translation Lookaside Buffer* or Translation Lookaside Buffer (TLB) cache. Although the TLB does not work quite like the other caches, the algorithmic effect is similar: it is faster to fetch two nodes from the same TLB block than from two separate blocks. Therefore, we can model the TLB as an additional level of cache in the hierarchy. The block size of the TLB cache is usually the page size of the computer (e.g. 4096 bytes), although larger blocks are sometimes also used.

In this article, we model the cache hierarchy as follows. There are k levels of cache, with block sizes $B_1 < B_2 < \dots < B_k$ (measured in bytes) at each level. In addition, we define B_0 to be the node size of the tree that is to be laid out in the cache, in bytes, and $B_{k+1} = \infty$. All nodes are assumed to have the same size. We assume that our algorithms know these cache parameters—in practice, they can be inferred by measuring the time taken by selected memory accesses, or from the CPU

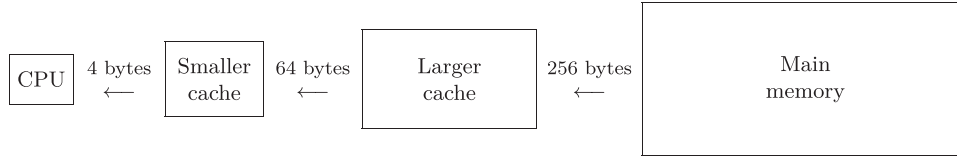


FIGURE 1. Reading a 4-byte word in a hypothetical memory hierarchy.

model or metadata stored in the CPU. Note that our model does not include cache sizes (only block sizes), since the algorithms that we consider do not have a use for them (the other cache-sensitive search trees discussed in Section 1 do not use cache sizes either). This also allows us to merge levels when they have the same block size, which is common, as hardware caches often use the same block size on several levels.

A concrete example of the cache hierarchy is given by one of the computers used for our experiments (Section 4). Its AMD Athlon XP processor has two levels of data caches: a 64 KB level 1 (‘L1’) cache and a 512 KB level 2 (‘L2’) cache. However, both caches have 64-byte blocks, so they only use one level in the hierarchy of the cache model. The TLB cache uses 4096-byte pages (a small number of 2 and 4 MB TLB blocks also exist in the TLB of this processor, but these were not used in the experiments). The parameters of the full hierarchy used in the experiments are thus:

$$k = 2$$

$$B_0 = 16 \quad (\text{the binary trees have 16-byte nodes})$$

$$B_1 = 64 \quad (\text{the block size of the L1 and L2 caches})$$

$$B_2 = 4096 \quad (\text{the page size of the TLB cache})$$

$$B_3 = \infty.$$

Our algorithms assume that, for $i > 1$, each block size B_i is an integer multiple of B_{i-1} ; this is always true in actual hardware caches, since the values of B_i with $i \geq 1$ are powers of 2. Additionally, if B_1 is not an integer multiple of the node size B_0 , a node should not cross a B_1 -block boundary, so that it is never necessary to fetch two cache blocks from memory in order to access a single node. To achieve this, the allocation of new nodes needs to be modified so that the last $(B_1 \bmod B_0)$ bytes of each B_1 -block are not used. The memory locations of cache-block boundaries are easy to infer from the block sizes: a memory address A points to the first byte of a level i cache block when $(A \bmod B_i) = 0$ ¹.

The main efficiency measure that we use to analyze search trees is the B_i -block search path length.

DEFINITION 2.1. *The B_i -block search path length, denoted as P_i , $0 \leq i \leq k$, is the length of a root-to-leaf path in a tree, measured in the number of separate cache blocks of size B_i encountered on the path.*

Thus, P_0 is the traditional search path length in nodes (assuming that the search ends at a leaf), P_1 is the number of separate B_1 -sized cache blocks encountered on the path, and so on. Traditional balanced search trees attempt to minimize P_0 (of a random search, i.e. averaged over all root-to-leaf paths); note that $P_{i-1} \geq P_i$, since each B_{i-1} -block is completely contained in a single B_i -block.

3. LOCAL RELOCATION

In this section, we discuss our modifications to the insert and delete algorithms that preserve a good memory layout without increasing the time complexity of insertion and deletion. Since we do not change the structure of the nodes, our approach only changes the locations of the nodes in memory—in other words, the memory layout of the tree. We name our approach *local relocation*, since the layout changes are localized to a few nodes around each structure modification.

3.1. The invariant

The memory-layout property that we will preserve is that each node in the tree has some neighboring nodes accompanying it on the same cache block. Specifically, we require that at least α (a tunable parameter) nodes are placed on the same cache block, such that they form a connected subgraph of the tree: that is, the block may contain any subset S of at least α nodes from tree T , as long as the path from any $s_1 \in S$ to any $s_2 \in S$ uses only nodes that are in S . To make preserving the invariant easier, we relax our condition so that any connected subgraph containing a leaf node is valid: subgraphs containing leaves have a proportionally small effect on the root-to-leaf paths of large trees.

Stated formally, we preserve the following memory-layout invariant.

Invariant 3.1. Each non-leaf node x of the search tree T is part of a connected subgraph of T that is stored in a single B_1 -sized cache block, and contains either at least α non-leaf nodes or at least one leaf node.

¹ For virtual addresses this assumes that the page size is an integer multiple of all cache block sizes, which is true since they are all powers of 2 and no larger than the page size.

This invariant only considers one level in the cache hierarchy. We concentrate on laying out small nodes in small (B_1 -sized) cache blocks; however, the technique also works with larger node and cache-block sizes, e.g. 64-byte nodes placed in 4096-byte TLB pages.

For a given ratio B_1/B_0 , there is an upper limit for the possible values of α . Obviously $\alpha \leq B_1/B_0$ (α nodes must fit in the cache block). However, our update algorithm given below further restricts this to $\alpha \leq (2 + B_1/B_0)/3$. (Thus in the particular architecture and node size used as an example above $B_1/B_0 = 4$, so α must be 2.)

Before giving the update algorithm, we examine the quality of this memory layout. Recall from Section 2 that we measure quality using the average B_1 -block path length, averaged over all root-to-leaf paths in the tree. We show that, even in the worst case, Invariant 3.1 improves this path length by a factor that depends on α , when compared with a worst-case tree without any layout invariant.

To keep the result independent of the balancing strategy of the underlying tree, we analyze complete trees, i.e. ones where all root-to-leaf paths have the same length. To see that Invariant 3.1 improves the memory layout of, e.g. a red-black tree, recall that the top part of a red-black tree of height h is a complete tree of height at least $h/2$ (in practice often close to h).

THEOREM 3.1. *Assume that a complete binary tree T of height $h \gg \alpha$ has been laid out in memory so that Invariant 3.1 is satisfied. Then the expected value of the B_1 -block path length of a random search in T is*

$$E[P_1] \leq \frac{2^{\alpha-1}}{2^{\alpha}-1}h + \frac{1}{2}.$$

Proof sketch. In a specific memory layout, $E[P_1]$ is counted over all root-to-leaf paths in the tree. Thus, an upper bound for $E[P_1]$ can be derived from a worst-case memory layout, in which each B_1 -sized cache block contains only the nodes prescribed by Invariant 3.1, and in a worst-case arrangement. Consider all possible arrangements allowed by Invariant 3.1,

i.e. the possible shapes of connected subgraphs of T with α elements.

Clearly, the largest B_1 -block path length is reached when each connected subgraph is a single chain of parent-child links, as in Fig. 2, so that no node has both of its children in its cache block. More formally, the worst-case layout we will examine is as follows: The root node r of T begins a chain C of α parent-child links (i.e. a path down from r to a node on level α). Any child of the nodes in C that is not in C starts a new chain in the same way, and each cache block used to store the tree contains (only) exactly one of these chains.

To calculate the B_1 -block path length, consider all possible paths down from a node p that starts a new chain C_p . Assume without loss of generality that C_p contains the left child of p . One half of all the paths from p descend left from p to the next node in C_p , while the other half (paths that descend to p 's right child) lead to the topmost node q of a new chain C_q , which is located on a separate cache block. In a complete binary tree, if p has height h , the height of q is $h-1$. Similarly, $\frac{1}{4}$ of the paths down from p lead to the third node in C (if $\alpha > 2$), and the remaining $\frac{1}{4}$ lead to a new chain at the root of a subtree with height $h-2$.

Let $P(h)$ denote the expected B_1 -block path length in a subtree of height h rooted at the topmost node of a chain. In general, $P(h)$ obeys the recurrence

$$P(h) = \begin{cases} (h+1)/2 & \text{for } h < \alpha, \\ 1 + \left[\sum_{i=1}^{\alpha} \frac{1}{2^i} P(h-i) \right] + \frac{1}{2^{\alpha}} P(h-\alpha) & \text{otherwise.} \end{cases}$$

The base case is found by examining a tree of height $h < \alpha$ and its worst-case layout into cache blocks: similar to the above, except that the chains of parent-child links can simply end at any leaf node (i.e. take only the topmost h levels of Fig. 2).

An induction proof shows that $P(h) \leq \frac{1}{2} + h \cdot 2^{\alpha-1}/(2^{\alpha}-1)$ is a solution to this recurrence. Thus, $E[P_1]$ [any memory layout satisfying Invariant 3.1] $\leq E[P_1]$ [this worst-case memory layout] $= P(h) \leq \frac{1}{2} + h \cdot 2^{\alpha-1}/(2^{\alpha}-1)$. $\square$

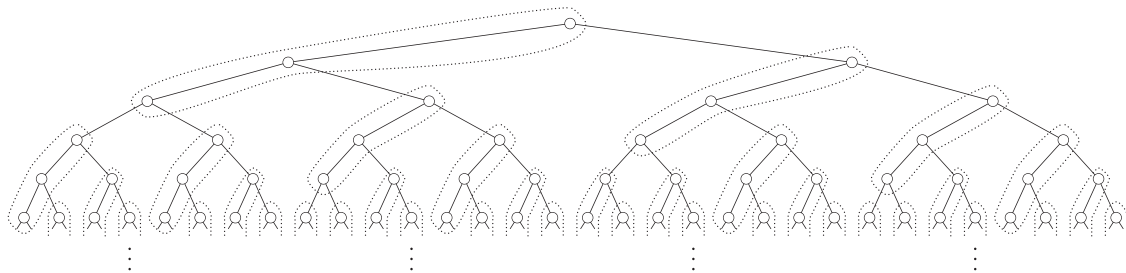


FIGURE 2. A worst-case arrangement of nodes that satisfies Invariant 3.1, here with $\alpha = 3$. The left-right order of the children (subtrees) is not essential: the layout could be made less regular by exchanging the children of any of the nodes. Any other change to this layout that preserves the invariant would necessarily move both children of some node p onto the same cache block with p , which would improve the memory layout of the subtree rooted at p without affecting the layout elsewhere in the tree.

Examining specific values of α , Theorem 3.1 gives $E[P_1] \leq \frac{1}{2} + 2h/3$ for $\alpha = 2$ (i.e. at least 33% improvement to the worst-case memory layout that does not satisfy Invariant 3.1), $E[P_1] \leq \frac{1}{2} + 4h/7$ for $\alpha = 3$ (42% improvement), $E[P_1] \leq \frac{1}{2} + 8h/15$ for $\alpha = 4$ (46% improvement). These values approach $\frac{1}{2} + h/2$ in the limit (when $\alpha \rightarrow \infty$), with rapidly decreasing improvement for larger values of α . This analysis points to small values of α being preferable, and even $\alpha = 2$ giving a 33% improvement over not using the invariant at all.

Why is $h/2$ the limit? Invariant 3.1 allows a connected subgraph of any shape to be placed in a cache block. It is thus possible, even with large α , to have a layout with every right child of every node located on a separate cache block from its parent, as in Fig. 2. This leads to at most 50% improvement over the worst-case non-cache-sensitive tree, which has each node on its own cache block.

To get further than $h/2$, the memory-layout invariant would additionally have to dictate the shapes of the connected subgraphs, e.g. by limiting their height to $O(\log \alpha)$. However, this would make preserving the invariant more complex to implement, as well as requiring the modification of a larger set of nodes to preserve the invariant after an insertion or deletion, which would increase the time complexity.

3.2. Preserving the invariant

We will call a node x *broken* if Invariant 3.1 does not hold for it. To analyze how this can happen, denote $N(x)$ = the set of *neighbors* of node x , i.e. the parent of x and both of its children (if they exist). Furthermore, say that x *depends* on y if moving (only) y to another cache block would make x broken.

Assume that the invariant initially holds in a binary search tree T . After any structure modification operation (actual insertion, actual deletion or rotation) is done in T , some of its nodes can potentially be broken. To re-establish the invariant, the algorithm given in the next section is executed after every rotation and after nodes are inserted or deleted. The algorithm is given a list of nodes (e.g. 1–6 nodes for $\alpha = 2$) that could potentially have been broken by the (single) structure modification.

For $\alpha = 2$, the nodes that can break in a structure modification are exactly those whose parent or either child changes, since a node will break if it depends on a node that is moved away or deleted.

Figure 3 shows all the cases where nodes can break. In external or leaf-oriented trees, insertion (a) always inserts a new leaf node y and its new parent x in place of a leaf node q , while deletion (b) deletes a leaf node y and its parent x . In both cases, the parent p of x may break, since its child changes. In (a), x itself may be initially broken (if it is not allocated in the same cache block as y); in (b), the remaining sibling node q of y (if any) may break, since its parent changes.

When inserting to an internal tree (c), the parent of the newly inserted node may break if it was previously a leaf. Deletion in internal trees has two cases. If a leaf is deleted (d), its parent

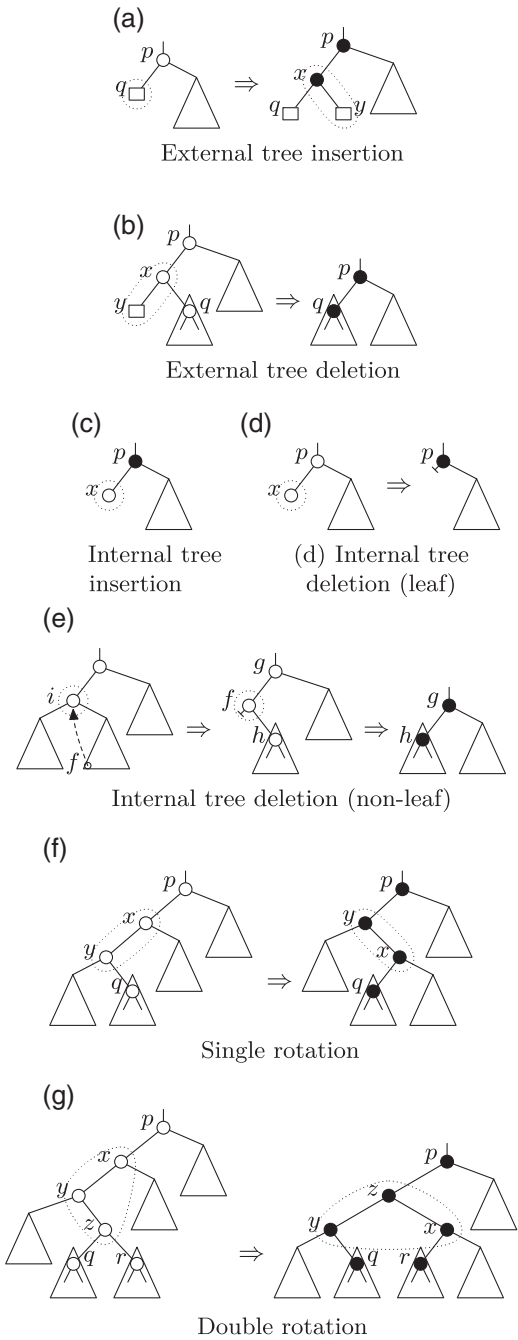


FIGURE 3. Broken nodes in actual insertion, actual deletion and rotations, for $\alpha = 2$. Potentially broken nodes are filled black; the dotted lines indicate the nodes that the operation works on. (a) External tree insertion, (b) External tree deletion, (c) Internal tree insertion, (d) Internal tree deletion (leaf), (e) Internal tree deletion (non-leaf), (f) Single rotation and (g) Double rotation.

may break (unless it becomes a leaf). Otherwise (e), deletion of an internal node i is done by searching down the tree for the ‘follower’ node f with the next-larger key (or equivalently the

next-smaller key), and replacing² the key and data fields of i with those of f . Node f with a parent g always has at most one child (call it h), so f can now be detached from the tree by setting the child pointer in g to point to h . Then both g and h may break, since f is no longer the child of g nor the parent of h .

Rotations (f) and (g) rotate the two or three nodes indicated by the dotted line in Fig. 3f and g into a different shape. Here, the parent p of the topmost rotated node may break, since its child changes. In addition, the children of the rotated nodes may break, except for the leftmost and rightmost children, whose parents do not change. Finally, the rotated nodes themselves may break, because their parents and children change.

When $\alpha > 2$, the same nodes can break as when $\alpha = 2$; call this initial set of potentially broken nodes R_0 . In addition, some other nodes on the cache blocks that contain nodes in R_0 can break; they are found as follows. For each potentially broken node $r \in R_0$, we find the neighbors of r that are on the same cache block as r . For each r , we continue this search until we have found at least α nodes, or any leaf, or all the remaining neighbors are on different cache blocks. If we have found α nodes or a leaf, then r is not broken. On the other hand, if less than α nodes and no leaves were found, then all of these nodes (i.e. a connected subgraph of the tree that contains node r and at most $\alpha - 2$ other nodes, all on the same cache block) are broken.

In summary, we have the following theorem.

THEOREM 3.2. *One structure modification can cause $O(\alpha)$ nodes to be broken. They can be found in time $O(\alpha)$.*

Proof. By the above discussion, there are at most six broken nodes in the case $\alpha = 2$ (the black nodes in Fig. 3). When $\alpha > 2$, each of the six nodes in the initial set R_0 can produce $\alpha - 2$ additional broken nodes in the worst case, for a total of $6\alpha - 6$. (These additional broken nodes account for, e.g. the case where a chain of α connected nodes gets disconnected in the middle. A smaller constant factor could be shown for instance by noting that node z in Fig. 3g cannot produce additional broken nodes.) $\square$

3.3. Fixing broken nodes for $\alpha = 2$

We now describe how to fix broken nodes, starting with the case $\alpha = 2$. Here we use an additional definition: $D(x)$ is defined to be the subset of the neighbors of node x that depend on node x (i.e. will be broken if x is moved to another cache block). Algorithm 1 uses the property that a broken node r can be moved freely, because $D(r) = \emptyset$: all of its neighbors $N(r)$ are on different cache blocks, as otherwise r would not be broken.

Each iteration in Algorithm 1 first removes any non-broken nodes from the set R of potentially broken nodes (initially R_0).

² A common alternate implementation adjusts child pointers to swap f and i , and then deletes f . However, this is obviously worse from a memory layout perspective, as confirmed experimentally by Oksanen and Malmi [11].

Then the first available one of three options is executed, and these two steps are repeated until R becomes empty.

The first option of the three examines the cache blocks of all neighbors of the broken nodes to find a neighbor x with free space in its cache block. If such a neighbor is found, a broken node $r \in N(x)$ is fixed by moving it to this cache block. If no such neighbor exists, the second option is to examine the cache blocks of the nodes in R : if one of them has enough space for a neighbor x and all of its dependants $D(x)$, they are all moved to this cache block.

Otherwise, the third option is to forcibly fix a broken node r by moving it and some neighboring nodes to a newly allocated cache block. At least one neighbor of r needs to be moved along with r to make r non-broken; but if a neighbor x was not broken, some of its other neighbors may depend on x staying where it is—these are exactly the nodes in $D(x)$, and they are all moved to the new cache block. It is safe to move the nodes in $D(x)$ together with x : since they depend on x , none of their other neighbors are on the same cache block.

THEOREM 3.3. *Algorithm 1 moves at most $4|R| = O(|R|)$ nodes in memory. The total time complexity of the algorithm is $O(|R|^2)$.*

Proof. Each iteration of the loop in Algorithm 1 fixes at least one broken node. Line 5 does this by moving one node; line 11 moves 2–4 nodes (r , x , and at most two other neighbors of x), and line 8 moves 1–3 nodes. There are at most $|R|$ iterations, and thus at most $4|R|$ nodes are moved.

Each iteration looks at $O(|R|)$ nodes, making the total time complexity $O(|R|^2)$, assuming that the amount of free space in a cache block can be found in constant time. However, a naïve implementation finds free space by looking at every node in the B_1 -block to locate free positions for nodes. This increases the time complexity to $O(|R|^2 \cdot \lfloor B_1/B_0 \rfloor)$, but may actually be preferable with the small B_1 of current processors. The implementation described in Section 4 did this, with $B_1/B_0 = 4$.

With larger B_1/B_0 , the bound of the theorem is reached by keeping track of the number of free positions for nodes in an integer stored in a fixed location inside the B_1 -sized block.³ To find a free node in constant time, a doubly linked list of free nodes can be stored in the otherwise unused free nodes themselves [11], with a pointer to the head of this list stored in a fixed location inside the B_1 -block. $\square$

3.4. Fixing broken nodes for $\alpha > 2$

The main difference in fixing broken nodes when $\alpha > 2$ is in the handling of dependants: when $\alpha = 2$ and a non-broken node x is moved to another cache block, the only nodes that need to be

³ A few bits are enough to store this: $\log_2 \lfloor B_1/B_0 \rfloor$ must be much smaller than, e.g. the size of a single child pointer.

ALGORITHM 1 The local relocation algorithm for $\alpha = 2$.FIX-BROKEN(R):

```

1 while  $R \neq \emptyset$  do
2   Remove any non-broken nodes from  $R$ , and exit if  $R$  becomes empty.
3   if a node in  $N(R)$  has free space in its cache block then
4     Select such a node  $x$  and a broken neighbor  $r \in R$ . Prefer the  $x$  with the most free space and an  $r$  with no broken neighbors.
5     Move  $r$  to the cache block containing  $x$ .
6   else if a node  $r \in R$  has enough free space in its cache block then
7     Select the neighbor  $x \in N(r)$  with the smallest  $|D(x)|$ .
8     Move  $x$  and all nodes in  $D(x)$  to the cache block containing  $r$ .
9   else
10    Select a node  $x \in N(R)$  and its broken neighbor  $r \in R$ . Prefer a broken  $x$ , and after that an  $x$  with small  $|D(x)|$ . If there are multiple choices for  $r$ , prefer one with  $N(r) \setminus \{x\}$  non-broken.
11    Move  $r$ ,  $x$  and all nodes in  $D(x)$  to a newly allocated cache block.

```

Note: R is a set of potentially broken nodes which the algorithm will make non-broken; $N(R) = \bigcup_{r \in R} N(r)$, where $N(r)$ is the set of neighbors of node r . An implementation detail is that the algorithm needs access to the parent, grandparent and great grandparent of each node in R , since the grandparent may have to be moved in lines 8 and 11. The search-tree algorithms can temporarily store the (partial) list of parents together with each node that they process; the list is initialized while descending down in the tree and needs to be updated in rotations, each of which changes the parents in a particular way. The changes occur close to the current nodes, so the parent-list updates are $O(1)$ per rotation.

moved with it are some of its neighbors (i.e. the set $D(x)$ defined above).

When $\alpha > 2$, the need to move neighboring non-broken nodes together may propagate beyond the immediate neighbors. If a non-broken node x needs to be moved to another cache block (to fix a broken neighbor of x that is also moved), each non-broken neighbor y of x that is on the same cache block as x may need to be moved as well (since it may depend on x staying where it is, as with $\alpha = 2$). In addition, each y can require at most $\alpha - 2$ other nodes to be moved along with it: these are connected to y using only nodes in the same cache block as both x and y . The maximum is $\alpha - 2$, since if y has $\alpha - 1$ other connected nodes on its cache block, then y does not depend on x .

Algorithm 2 fixes broken nodes by first selecting from the set of broken nodes a connected subset of 1 to α broken nodes (line 2). Neighboring non-broken nodes are then added to this set (lines 3–10) to make it large enough to be moved to its own cache block (i.e. until it contains either α nodes or a leaf). If there is free space in one of the existing cache blocks containing the nodes involved, the set is moved there (line 13); otherwise, a new cache block is allocated (line 18). This process fixes the broken nodes that were selected and never breaks any; it is repeated until all nodes have been fixed.

Adding neighboring nodes to the selected set U (lines 3–10) can be done as follows. First, we select an arbitrary neighbor v of a node u in U and add v to U (lines 4–5; on the first iteration, v is always non-broken). However, if v is non-broken, some of its neighbors could break if v was moved to another cache block. We resolve this by including all such nodes in U as well (lines 7–10)—they are found by looking at the other neighbors of v that are on the same cache block as v . For each such neighbor w , we search, staying on the cache block that contains v , for a connected subgraph that contains any leaves or α nodes

(including w but excluding v). If a large enough subgraph is found, moving v will not break w and we are done; otherwise, we include the subgraph that we found in U (line 10).

After U is moved, all nodes in U are non-broken. In addition, moving nodes to an existing cache block can also fix some other nodes that are in this block. We remove these from R if necessary (lines 14–16).

THEOREM 3.4. *Algorithm 2 moves at most $O(\alpha|R|)$ nodes in memory. The total time complexity of the algorithm is $O(\max\{\alpha^2|R|, \alpha|R|^2\})$.*

Proof. On each iteration of the outermost loop, U initially (after line 2) contains 1 to α broken nodes—in the worst case only one. Lines 3–10 are executed until U has α nodes (or any leaves). Lines 5–10 add 1 to $1 + 2(\alpha - 1)$ nodes to U (since $|N(v) \setminus \{u\}| < 3$ and, on line 10, $|W| < \alpha$). Thus, the largest $|U|$ is reached when $|U| = \alpha - 1$ at line 3, and then U can finally contain at most $3\alpha - 2$ nodes, only one of which is broken in the worst case. Therefore, in the worst case $3\alpha - 2$ nodes are moved to fix one broken node, and a total of at most $|R| \cdot (3\alpha - 2) = O(\alpha|R|)$ nodes are moved to fix all nodes in R . (This worst case is behind the restriction $\alpha \leq (2 + B_1/B_0)/3$ mentioned in Section 3.1: $3\alpha - 2$ nodes must fit in a cache block for line 18.)

Line 2 uses $O(\alpha)$ time. Lines 3–10 use $O(\alpha^2)$ time, since lines 6–10 can use $O(\alpha)$ time without adding any nodes to U (the time is used to check that v can be moved without its neighbors). Lines 12–18 use $O(|U|) = O(\alpha|R|)$ time if the amount of free space in a cache block can be found in constant time. Since each iteration of the outer loop could fix only one broken node, the total time complexity is $O(|R| \max\{\alpha^2, \alpha|R|\}) = O(\max\{\alpha^2|R|, \alpha|R|^2\})$.

ALGORITHM 2 The local relocation algorithm for $\alpha > 2$.

 FIX-BROKEN-LARGER-ALPHA(R):

```

1  while  $R \neq \emptyset$  do
2     $U \leftarrow$  an arbitrarily selected connected subgraph containing only nodes from  $R$ , as large as possible with  $|U| \leq \alpha$ 
3    while  $|U| < \alpha$  and  $U$  does not contain leaves do
4       $(u, v) \leftarrow$  an arbitrary pair of neighboring nodes with  $u \in U$  and  $v \notin U$ 
5       $U \leftarrow U \cup v$ 
6      if  $v \notin R$  then
7        for  $w \in C_v \cap N(v) \setminus \{u\}$  do
8           $W \leftarrow$  a connected subset of  $C_v \setminus \{v\}$  that contains  $w$ , as large as possible with  $|W| \leq \alpha$ 
9          if  $|W| < \alpha$  and  $W$  contains no leaves then      {  $w$  would break }
10          $U \leftarrow U \cup W$ 
11     $R \leftarrow R \setminus U$ 
12    if a node  $x \in U$  has space in  $C_x$  for all nodes in  $U$  then
13      Move all nodes in  $U$  to  $C_x$ . Prefer the node  $x$  with the most free space in  $C_x$ .
14      for  $r \in R \cap C_x$  do
15        if  $r$  is now non-broken then
16           $R \leftarrow R \setminus \{r\}$ 
17    else
18      Move all nodes in  $U$  to a newly allocated cache block.
    
```

Note: Here the argument R is the initial set of actually broken nodes, as described at the end of Section 3.2. C_x is (the set of nodes in) the cache block containing node x . An implementation detail left out for clarity is that the algorithm needs to keep track of a linked list of parents for each node that it uses.

Finding free space in the cache blocks can be done in the same way as in Theorem 3.3. $\square$

Since visiting an uncached memory area is the most costly operation in a cache-sensitive algorithm, we should also look at the number of cache blocks that the algorithms need to transfer from memory. Here, the algorithms are optimal (disregarding constant factors and assuming that the cache can hold all the required blocks).

THEOREM 3.5. *Algorithms 1 and 2 visit at most $O(|R|)$ separate cache blocks.*

Proof. Both algorithms only look at the cache blocks of broken nodes and their immediate neighbors. In addition, one new cache block can be allocated per broken node. $\square$

The previous complexity results were presented in terms of $|R|$, the number of broken nodes given to Algorithm 1 or 2. Since $|R| = O(\alpha)$ (Theorem 3.2), we can now summarize from the above theorems.

THEOREM 3.6. *After a single structure modification, Algorithms 1 and 2 can be used to re-establish Invariant 3.1 in time $O(\alpha^3)$. The algorithms move $O(\alpha^2)$ nodes in memory, visiting $O(\alpha)$ separate cache blocks.*

Recall that α is a tunable parameter, and small values of α are preferable—so all of these are $O(1)$ for fixed α .

3.5. Additional optimizations

Though it is not required for Invariant 3.1, the memory layout can be optimized somewhat further with a simple heuristic: in insertion, a new node should be allocated in the cache block of its parent if it happens to have enough free space.

For $\alpha = 2$ in an internal AVL tree, broken nodes can be fixed in a simpler way in the special case where a leaf node is deleted (Fig. 3d). If the leaf node n and its parent p are on the same cache block, and the other child of p (call it n') is on a different cache block, then move n' to the location where n was deleted from (in an AVL tree, n' must be a leaf). Otherwise—if p and n are on different cache blocks, or if n' does not exist or is on the same cache block as n —nothing needs to be done.

3.6. Space usage

Local relocation includes a space–time tradeoff: it sometimes allocates a new cache block to get α nodes on the same cache block (thus improving cache locality), even though existing cache blocks would have space for the nodes. Since the algorithms always prefer unused locations in a previously allocated cache block, it is to be hoped that the cache blocks do not become very empty on average. We examine the space usage empirically in Section 5.

Instead of moving nodes to a new cache block, it would in some cases be possible to re-establish the invariant by rearranging all nodes on the existing cache blocks that are under consideration. However, moving unrelated nodes on the existing cache blocks ‘out of the way’ of the currently broken nodes is not practical: moving a node x in memory needs access to its parent to

TABLE 1. CPU and cache parameters of the four computers used in the experiments. The Atom computer has one physical CPU core but uses hyper-threading to provide two logical cores. All four had the same block size, 64 bytes, in both the levels 1 and 2 data caches, and a TLB page size of 4096 bytes.

CPU model	Cores	Speed (MHz)	Level 1 data cache	Level 2 data cache
AMD Athlon XP	1	2167	64 KB, 2-way set-associ.	512 KB, 16-way set-associ.
AMD Sempron 3000+	1	1800	64 KB, 2-way set-associ.	128 KB, 16-way set-associ.
Intel Atom N270	1 + <i>HT</i>	1600	24 KB, 6-way set-associ.	512 KB, 8-way set-associ.
Intel Core2 Quad	4	2660	32 KB, 8-way set-associ.	3 MB, 12-way set-associ.

update the link that points to x . But there is no simple way to find the parent, unless the nodes contain parent links, which we do not want to require because of their overhead.

A lower limit for the cache block fill ratio can be obtained from the property that local relocation preserves: each cache block contains either a leaf node or at least α non-leaf nodes. Empty cache blocks should of course be reused by new allocations.

3.7. Fixing broken nodes less frequently

In theory, it would be possible to execute local relocation less frequently than was done above. For instance, Algorithm 1 or 2 could be run after a full sequence of rotations after an insertion or deletion, and not after each individual rotation. In this way fewer nodes might need to be moved, since a larger number of broken nodes gives more freedom in how to re-establish the invariant. Also, fixing broken nodes after individual rotations has the disadvantage that the very next rotation may break some of the nodes that were just carefully fixed.

However, Algorithms 1 and 2 take time proportional to the square of the number of broken nodes (Theorems 3.3 and 3.4). This approach would thus increase the total worst-case time complexity of insertion and deletion, which we wanted to avoid.

4. EXPERIMENT SETUP

For our experiments, we implemented the algorithms of Section 3 on internal AVL and red-black trees, with $\alpha = 2$ as discussed in Section 3.1. We include traditional non-cache-sensitive AVL and red-black trees for comparison. We also compared the binary search trees with cache-sensitive B^+ -trees (specifically our reimplementations of the ‘full CSB^+ -tree’ of [1]) and to standard B^+ -trees with cache-block-sized (B_1 -sized) nodes—the latter is called a ‘ cB^+ -tree’ below.

Since no cache-sensitive methods other than our own are known for dynamic AVL and red-black trees, we compare our method with a static approach that rearranges all nodes into a good cache-sensitive memory layout without regard for preserving this layout in insertions and deletions. Several cache-sensitive tree layout algorithms exist in the literature [4, 11, 12];

our own, called global relocation [13, 14], is more specifically designed for balanced search trees and includes some useful optimizations. It is multi-level, here laying out nodes into optimal B_1 -sized blocks, which are laid out in B_2 -sized blocks. We also apply this global relocation algorithm to the B-trees.

As an additional reference for comparison, we include a static cache-oblivious layout: with the block sizes fixed to $B_i = (2^{2^i} - 1) \cdot B_0$ ($1 \leq i \leq 4$ is enough for our trees), global relocation produces a layout that is very close to the van Emde Boas layout used in cache-oblivious search trees [13, 14]. While not a dynamic algorithm, this gives us a rough idea of how a cache-oblivious binary search tree (that does not alter the structure of the nodes) would function.

4.1. Details

We ran the experiments on four computers, whose CPU and cache parameters are given in Table 1. As was noted in Section 2, the cache parameters were as follows: $k = 2$, $B_0 = 16$, $B_1 = 64$, $B_2 = 4096$, $B_3 = \infty$. All the computers happened to have the same cache block sizes. Each individual experiment was repeated 15 times; all values given here are averages of these.⁴

In addition to search, insertion and deletion times, we also measured the average search path lengths in nodes, B_1 -blocks and B_2 -blocks (all four computers gave the same values, since the block sizes were the same). These were counted by examining all possible paths of successful searches in the tree—i.e. in the internal trees we included search paths that end before the leaf level. For internal trees, this gives slightly smaller path lengths than Definition 2.1 (which considers only paths that end at leaves), but we use this value in our experimental results, since it more closely measures the actual number of cache blocks visited by a successful search, and makes comparisons between internal and leaf-oriented trees more accurate. We name the values P'_1 and P'_2 to distinguish them from Definition 2.1.

⁴ All implementations were written in C, compiled as 32-bit code using the GNU C compiler (version 4.1.1 on AthlonXP, 4.4.5 on the others), and ran under the Linux kernel (version 2.6.18 on AthlonXP, 2.6.32 on the others). The TPC-C experiments used newer versions (GCC 4.7.2 and Linux 3.2.54). Cache parameters were found using Dave Jones’s x86info program.

The binary tree node size $B_0 = 16$ bytes was reached by using 4-byte integer keys, 4-byte data fields and 4-byte pointers to the left and right children. The AVL tree balance and red-black tree color information was encoded in the otherwise unused low-order bits of the child pointers. To save space, we did not use parent links in any of the tree implementations—rebalancing was done using an auxiliary stack. Only one of our test computers was 64-bit, so we used 32-bit pointers in all our experiments to make the results comparable.

To control the cache-sensitive placement of nodes and to avoid the space and time overhead of the standard `malloc` function, we always allocated our (fixed-size) nodes from a large array using a simple custom allocator. The large array was aligned to the largest cache block size B_2 .

The nodes of the B-trees were structured as simple sorted arrays of keys and child pointers (key and data fields in leaves, all 4-byte). The branching factor of a non-leaf node (with size $B_1 = 64$ bytes) was 7 in the cB^+ -tree and 14 in the CSB^+ -tree.

4.2. Using the TPC-C database benchmark

In addition to experiments on synthetic operations, we used more realistic operation sequences extracted from the well-known TPC-C database benchmark [10]. The benchmark models a real-world use case of large search trees (although normally disk-based ones) in databases, which we think makes it good for evaluating general-purpose search trees in a heavy-duty application. Further realism is added by gathering the operations from a real database management system. The operations interleave sequential and random reads and writes in specific parts of the full key range with some more complex access patterns.

The operation sequences were extracted as follows. We instrumented the InnoDB storage engine of MySQL version 5.5 to create a trace of its B-tree operations (InnoDB uses B-trees for all indexes), and ran a TPC-C benchmark.⁵ The benchmark first bulk-loads the initial contents of the tables using consecutive insertions, after which it executes randomly selected relatively short transactions. The latter are divided into a brief ‘rampup’ period (used for warming up buffers etc.) and a so-called ‘steady-state’; the results we give are from this steady-state.

The benchmark uses a single parameter for scaling up: the number of simulated warehouses. We used values ranging from 4 to 25, giving trees with 2.5 to 15 million keys.

We wanted to use each tree as a dense index, i.e. non-key data are to be stored outside of the tree (simulating the situation that leaf nodes contain pointers to these data, although we did not actually store the non-key data). We needed to post-process the produced trace of operations as follows: (i) MySQL uses

delete-marks, which we converted to actual deletion operations, and buffers some secondary-index modifications briefly in an ‘insert buffer’—we instrumented the operations before this buffering. (ii) MySQL does most of its operations using cursors, which we avoided (for instance, by converting a search followed by a deletion of the key at the cursor to a single key-based deletion). (iii) We converted row-update operations that do not affect the key value to searches (because non-key updates do not modify a dense index). Thus our final operation traces consist only of searches, insertions and deletions. (iv) We condensed complex multi-part keys into a single 32-bit integer—this allows directly comparing the results with our other experiments. The equality and order of all keys were preserved, so the logical structure of the tree was not affected. (v) We used a single tree to store all 11 indexes (9 primary and 2 secondary), by prefixing each key with the ID of the index. (Although some database management systems use this approach, MySQL uses the more common method of having a separate tree for each index.) We also ignored operations on the (extremely small) system tables MySQL uses to store metadata.

In addition to using the TPC-C benchmark operations as is, we made two modifications, shown separately in our results. The unmodified benchmark constructs most of the search tree using consecutive insertions in the initial setup phase. For B-trees, this is the worst case in terms of space usage (even worse for CSB^+ -trees, as we will see). Real-world B-tree implementations often include an ad hoc fix for consecutive insertions,⁶ to avoid this complication, we shuffled the initial insertions produced by the benchmark into random order (all permutations equally likely). Other operations, including all of the steady-state phase whose time we measure, were not changed. The steady-state phase of the benchmark includes small bursts of consecutive insertions, but small enough that they should not affect the results.

In addition, we wished to measure the efficiency of search operations using a realistic distribution of keys. To this end, we changed all (non-initial) operations in the TPC-C benchmark to searches, and show these results separately.

5. RESULTS FROM EXPERIMENTS

Since the purpose of cache-sensitive memory layouts is to improve search performance, we begin this section with our synthetic experiments on searches. Then we discuss insertions and deletions, and how they affect future searches. Finally, we show results from the realistic operation mix of the TPC-C experiments.

⁵ Specifically, OSDL DBT-2 from <http://osdl.dbt.sourceforge.net/>, which is an open-source implementation of the TPC-C benchmark.

⁶ For instance, the B-tree in MySQL’s InnoDB splits a node before it is full when it detects consecutive insertions, and sets the split position very unevenly, so that the half where future consecutive insertions would go is almost empty.

5.1. Search time, path length and space usage

For the experiments reported in Fig. 4 and Table 2, the tree was first initialized by inserting the keys $1, \dots, n$ in a random order (with each permutation equally likely) using single insertions. Then the cache was ‘warmed up’ with 10^4 successful searches of random keys (uniformly distributed in the range $[1, n]$). Finally, we measured the time taken by 10^5 searches, again of random keys that were present in the tree.

The same sets of operations were performed on different kinds of search trees with and without the relocation algorithms.

Results are shown in Fig. 4 and Table 2; the table gives numbers for the rightmost data point in the figures ($n = 10^7$). The AVL trees are not shown in the figures, since they performed almost identically to red–black trees, as is seen in Table 2.

It can be seen from Table 2 and the figures that local relocation improved search times in red–black and AVL trees by 26–32% on all four computers when the trees had $n = 10^7$ keys. With smaller trees, search times were improved less, but even with $n = 10^5$ there was still clear improvement over the non-cache-optimized trees. For instance, local relocation improved search

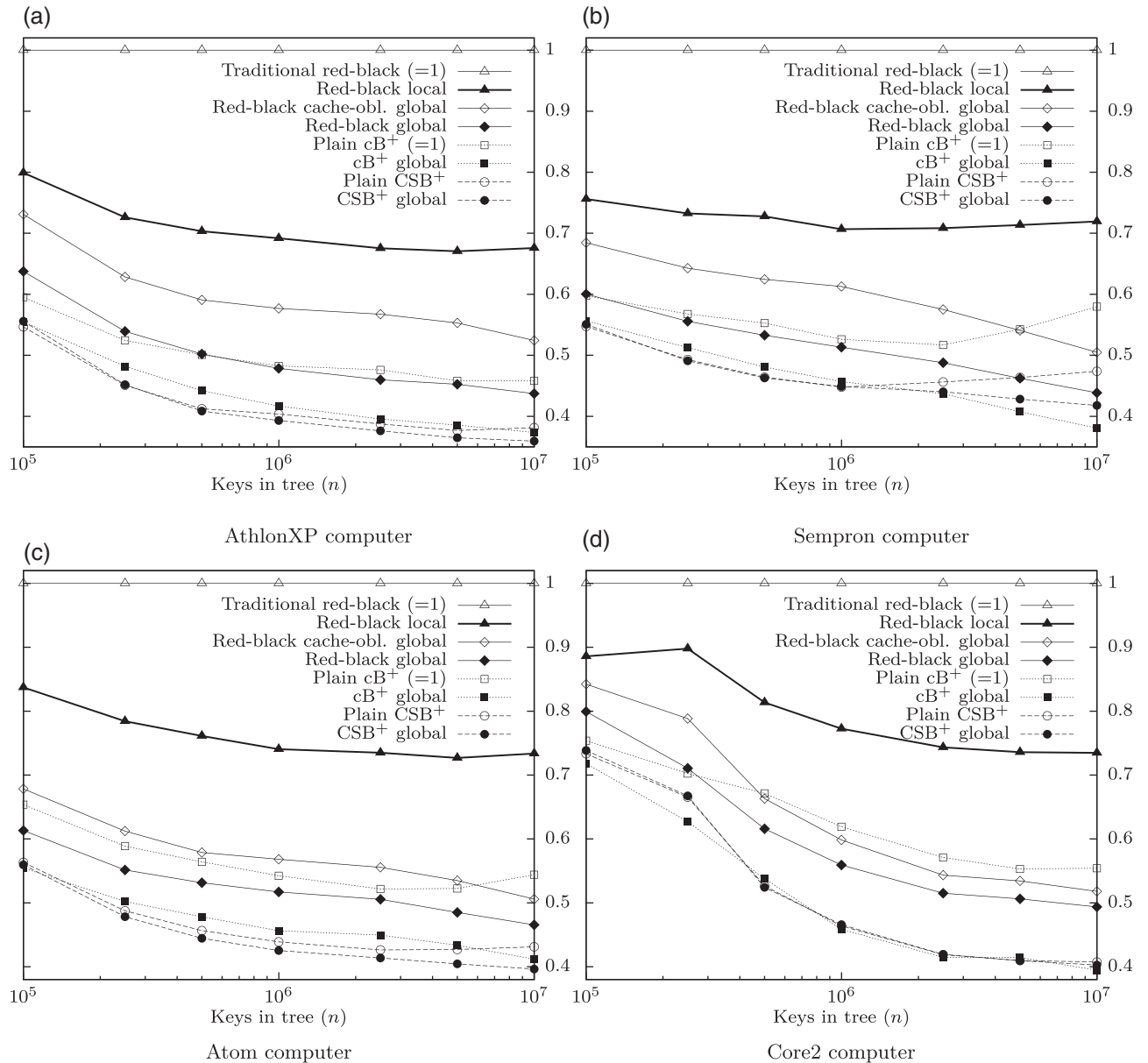


FIGURE 4. Search times using different trees and layout algorithms. The figures give the search time relative to the traditional red–black tree, on four computers. The trees marked ‘global’ have been relocated using the global algorithm, and ‘red–black local’ uses local relocation; the others use neither global nor local relocation. (a) AthlonXP computer; (b) Sempron computer; (c) Atom computer; (d) Core2 computer.

TABLE 2. Search times, average path lengths and memory used by the tree for various trees and relocation algorithms, all with $n = 10^7$ keys.

	Search time (nanoseconds per search)								Avg. path length			Size MB
	AthlonXP		Sempron		Atom		Core2		Nodes	P'_1	P'_2	
Traditional red-black	2327	100%	1543	100%	2246	100%	1089	100%	22.8	22.6	18.1	153
Red-black local	1572	68%	1110	72%	1648	73%	801	73%	22.8	13.7	12.8	176
Red-black global cache-obl.	1220	52%	779	50%	1136	51%	564	52%	22.8	11.9	6.2	216
Red-black global	1017	44%	677	44%	1046	47%	538	49%	22.8	10.6	3.4	173
Traditional AVL	2314	100%	1537	100%	2270	100%	1087	100%	22.7	22.5	18.0	153
AVL local	1568	68%	1113	72%	1650	73%	800	74%	22.7	13.7	12.4	177
AVL global cache-obl.	1214	52%	776	50%	1132	50%	562	52%	22.7	11.9	6.2	216
AVL global	1014	44%	675	44%	1042	46%	536	49%	22.7	10.5	3.4	174
Plain cB^+	1067	100%	895	100%	1222	100%	604	100%	9.0	9.0	8.5	149
cB^+ global	869	81%	587	66%	925	76%	430	71%	9.0	9.0	3.0	192
Plain CSB^+	888	100%	731	100%	969	100%	444	100%	7.1	7.1	7.0	206
CSB^+ global	836	94%	645	88%	890	92%	439	99%	7.1	7.1	5.1	206

The trees marked ‘global’ have been relocated using the global algorithm, and the ones marked ‘local’ use local relocation; the others use neither. The search times are also given in percentages relative to the traditional algorithms.

time by 22–33% for $10^6 \leq n \leq 10^7$, and even for $n = 10^5$ 10–11% on Core2 and 16–25% on the other computers.⁷

The static global relocation algorithm improved search times in the binary trees by 51–56%, so that the improved search times were close to the plain cB^+ -tree, somewhat better for large trees. The ‘cache-oblivious’ layout produced by global relocation was somewhat worse than the corresponding cache-sensitive layout, but a lot faster than a non-cache-optimized tree (dependent on n and the computer used, but 48–50% improvement for $n = 10^7$ in Table 2).

The percentage improvements are more or less in line with the changes in the B_1 -block path length (P'_1 in Table 2), which for $n = 10^7$ became 39% smaller with local relocation and 53% with global relocation. (This relation is more clearly visible in Fig. 6, to be presented in Section 5.3.) The path lengths also support the result of Theorem 3.1, which predicted that local relocation would improve the worst-case layout by at least 33%; note that building the traditional binary search trees by random insertions produces a layout that is nearly worst-case, as Table 2 shows (P'_1 is close to the path length in nodes).

The figures and Table 2 show that global relocation also improved search time in the B-trees. As can be seen in the figures, the four computers gave somewhat varying improvements for the B-trees; combining the values from all four in Table 2, cB^+ -trees were improved by 19–34% and CSB^+ -trees by 1–12% with $n = 10^7$.

Table 2 also gives the amount of memory occupied by the various trees (in megabytes or 2^{20} bytes) after the $n = 10^7$ insertions and possible relocation. In the binary trees, local

relocation increased memory usage by 15–16% and global relocation by 10–18% (for all tree sizes from 10^5 to 10^7 keys).

5.2. Insertion and deletion time

We also examined how using local relocation affects the running times of insertion and deletion. In these experiments, the tree was first initialized with n random insertions, and then $10^4 + 10^5$ random insertions or deletions were performed (on uniformly distributed locations in the tree). The times given are averaged from the 10^5 updates (the 10^4 were used to ‘warm up’ the cache). Results are given in Fig. 5.

On the AthlonXP computer shown in Fig. 5a and e, using local relocation increased insertion times by 8–46% for $10^6 \leq n \leq 10^7$. Deletions were affected less: local relocation made deletions ~15% faster to 5% slower for $10^6 \leq n \leq 10^7$. The Core2 computer had similar values (10–50%; 11% faster to 9% slower), but, for some reason, the other two computers were less efficient: in Sempron, insertions 35–95% slower and deletions 0–32% slower; in Atom, insertions 48–115% slower and deletions 4–43% slower.

Deletions were faster than insertions even without local relocation, but especially when it was used. This may be explained by the fact that random deletions from binary search trees produce fewer rotations than random insertions (see, e.g. [15]), so local relocation needs to examine fewer potentially broken nodes. Although local relocation takes time, it also decreases the time needed to search for the key to be inserted or deleted, which explains how deletions could sometimes be faster when using local relocation than without it.

The original CSB^+ - and CST -tree experiments [1,5] also displayed faster deletions than insertions. However, their

⁷ A tree with only $n = 10^5$ 16-byte nodes—1.5 MB without relocation, 1.8 MB when using local relocation—fits completely in the 3 MB L2 cache on the Core2 computer but not in the others.

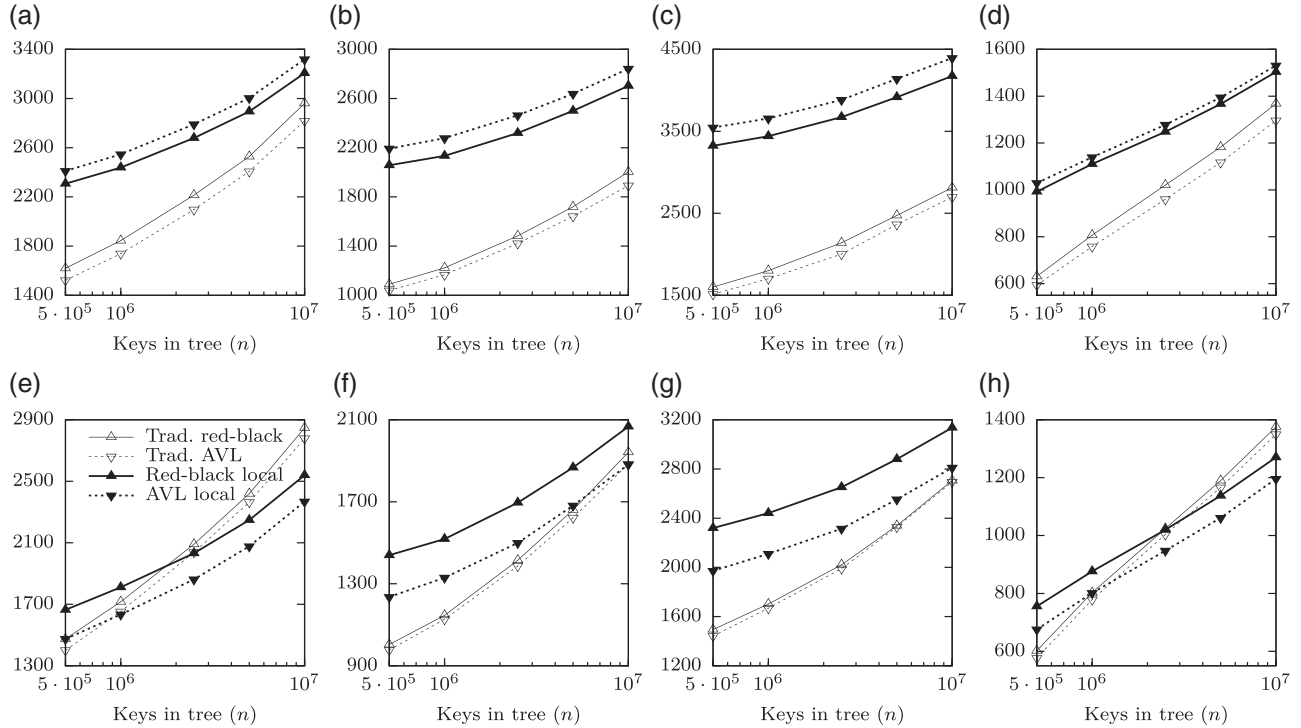


FIGURE 5. Effect of local relocation on the time taken by (a–d) insertions and (e–h) deletions. The times on the y axes are in nanoseconds per insertion or deletion; for clarity, the plot legend is shown only in (e). (a) Insertion, AthlonXP; (b) Insertion, Sempron; (c) Insertion, Atom; (d) Insertion, Core2; (e) Deletion, AthlonXP; (f) Deletion, Sempron; (g) Deletion, Atom and (h) Deletion, Core2.

situation was different since they both only marked keys as deleted instead of actually removing them from the tree. We used non-lazy deletion as is standard in binary search trees.

5.3. Degradation of locality after updates

As described above, global relocation can be used to rearrange all nodes of a tree into a good multi-level layout. A natural question to ask is how well such a layout is preserved when insertions and deletions are performed, either without regard to the cache or when local relocation is used to retain some of the cache-sensitivity.

In the experiment presented in Fig. 6, the tree was initialized with $n = 10^6$ random insertions. Then the global relocation algorithm was run once, and a number of random updates (half insertions and half deletions) were performed. Finally, the average search time from 10^5 random searches was measured (after a warmup period of 10^4 random searches), along with the average B_1 - and B_2 -block path lengths at the end of the experiment. Before $\sim 10^5$ updates after relocation (10% of the keys in the tree), the values changed very little; for clarity, the x -axis in Fig. 6 begins from 10^3 updates.

The results indicate that the cache-sensitivity of the tree decreases significantly only after roughly n updates have been performed (we also observed similar behavior using

other values of n in the range 10^5 to 2.5×10^6). Though local relocation does not quite match the cache efficiency of global relocation, it maintains a clearly better memory layout than if traditional non-cache-sensitive updates were used, thus allowing the expensive global relocation algorithm to be run less often.

5.4. Realistic operations from the TPC-C benchmark

We show results from the TPC-C experiments on two of our four computers (Atom and Core2), since the other two did not have enough memory to complete the larger operation sequences.

As noted in Section 4.2, in addition to using the TPC-C benchmark unmodified, we made two changes: initial insertions were made non-consecutive by shuffling them into a random permutation, and all operations were converted to searches. Figure 7 and Tables 3 and 4 show our results separately for all three cases.

The results of the ‘Searches only’ variation are directly comparable with those of Fig. 4 and Table 2: this searches-only TPC-C experiment is otherwise similar to the synthetic data but now the search operations are distributed realistically (according to the TPC-C benchmark) instead of being uniformly randomly distributed. With the TPC-C data, all search

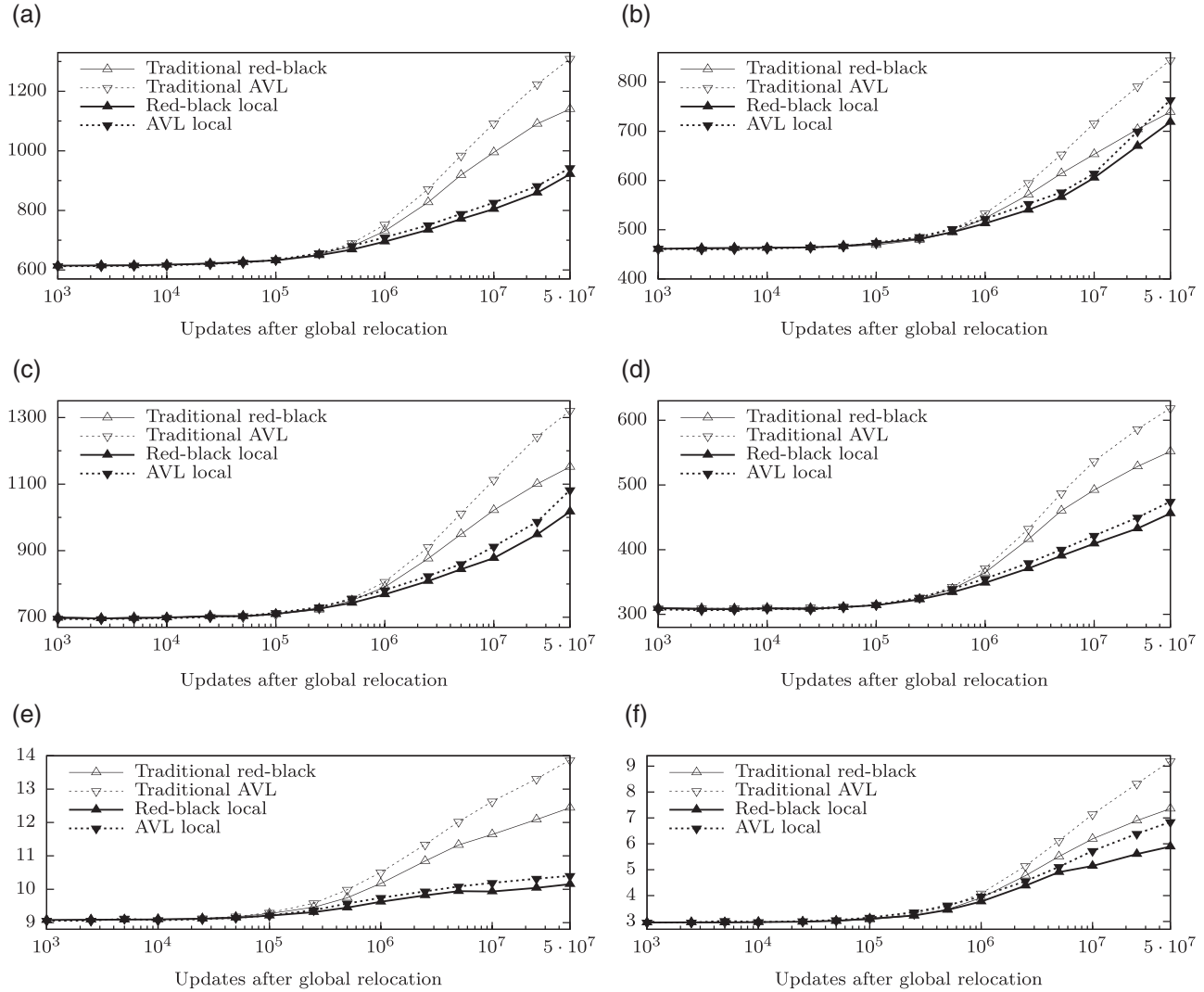


FIGURE 6. Degradation of locality when random insertions and deletions are performed after global relocation of a tree with $n = 10^6$ initial keys: (a–d) average search time from 10^5 searches in the different computers, (e) B_1 -block path length, (f) B_2 -block path length. (a) Search time (ns), AthlonXP; (b) Search time (ns), Sempron; (c) Search time (ns), Atom; (d) Search time (ns), Core2; (e) Average P'_1 (B_1 -sized blocks), all four computers; (f) Average P'_2 (B_2 -sized blocks), all four computers.

times are two to three times as fast (compare the two rightmost columns of Table 3 with Table 2), naturally since realistic data exhibits much more locality than random insertions. However, the relative efficiency of the relocation algorithms changes very little; compare the percentages in Table 2 with those of Table 3. Therefore it appears that, for all of these relocation algorithms, the synthetic results on random searches generalize well to realistic non-uniform search patterns. In particular, these TPC-C search patterns confirm the main result given above related to local relocation: that it improved search times by 26–32% (here 26–30%).

As discussed in Section 4.2, the ‘Shuffled insertions’ variation gives the most realistic results for the overall TPC-C

benchmark. Most of the results are more or less the same as in the ‘Searches only’ variation (most of the operations are searches), and so also close to the results on random searches in Section 5.1, although relocation usually gives a slightly smaller improvement. However, the local relocation algorithm performs differently: here it gives only 8–9% improvement for the Core2 computer and none for Atom. The reason is that insertions using local relocation take more time especially on Atom (as also seen in Fig. 5). Some of this is likely due to our prototype implementation of Algorithm 1 (not well optimized for its running time), but in general it appears that local relocation is most useful when a very large proportion of the operations are searches (more than the 91% in the TPC-C benchmark).

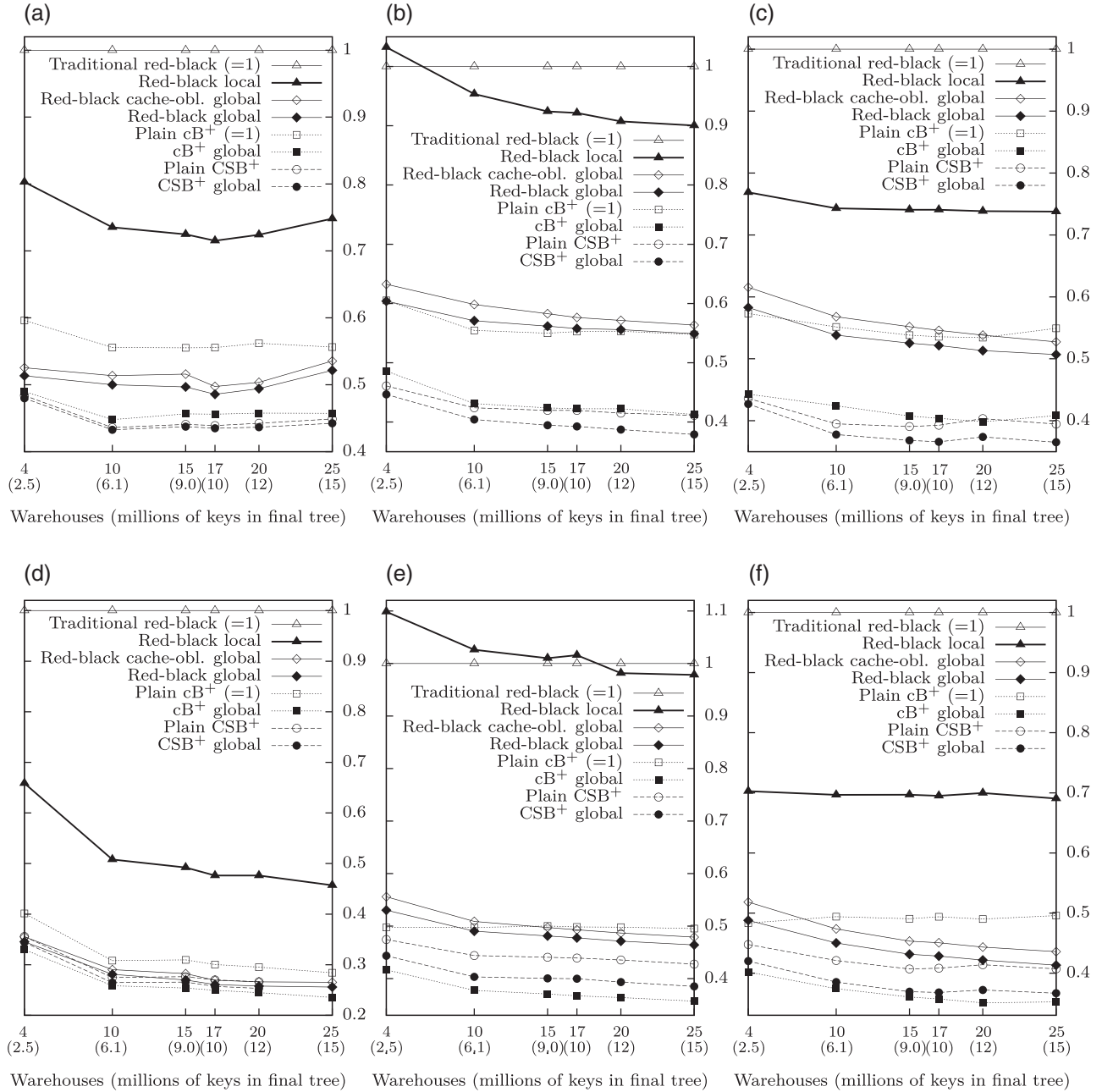


FIGURE 7. Results of experiments on the TPC-C benchmark: (a, d) the unmodified benchmark, (b, e) the benchmark with initial insertions shuffled, (c, f) the benchmark with initial insertions shuffled and all other operations converted to searches. The figures give the time taken by the steady-state phase of the benchmark, relative to the traditional red-black tree. Algorithms are named as in Fig. 4. The rightmost data point for CSB⁺-trees in (d) is missing due to the computer running out of memory. (a) Unmodified, Core2; (b) Shuffled, Core2; (c) Searches only, Core2; (d) Unmodified, Atom; (e) Shuffled, Atom; (f) Searches only, Atom.

Because ours is the first experiment to consider cache-sensitive search trees using real-world data, it is interesting to consider also the performance of the B-trees. Again the results were more or less the same as for random searches, with slightly less improvement when insertions and deletions

are included; e.g. on Core2, CSB⁺-trees improved cB⁺-trees by 27% (TPC-C searches only), 26% (random searches) or 24% (TPC-C shuffled ins.), consistent with the ‘more than 25%’ improvement reported by Rao and Ross [1] on random operation sequences with smaller trees in 2000.

TABLE 3. Results of experiments on the TPC-C benchmark, for 17 warehouses, including the unmodified benchmark, the benchmark with initial insertions shuffled, and one with initial insertions shuffled and all non-initial operations converted to searches.

	Time of steady-state phase (seconds)												ns per search	
	Unmodified				Shuffled ins.				Searches only				Searches only	
	Atom		Core2		Atom		Core2		Atom		Core2		Atom	
	Atom	Core2	Atom	Core2	Atom	Core2	Atom	Core2	Atom	Core2	Atom	Core2	Atom	Core2
Traditional red-black	20.38	100%	4.88	100%	11.31	100%	4.33	100%	10.56	100%	4.11	100%	929	361
Red-black local	9.71	48%	3.49	72%	11.49	102%	3.99	92%	7.34	70%	3.04	74%	646	268
Red-black global cache-obl.	5.49	27%	2.43	50%	5.57	49%	2.50	58%	4.76	45%	2.24	55%	419	197
Red-black global	5.31	26%	2.37	49%	5.40	48%	2.42	56%	4.52	43%	2.14	52%	398	188
Traditional AVL	18.09	100%	4.41	100%	10.71	100%	4.19	100%	10.41	100%	4.09	100%	916	360
AVL local	9.02	50%	3.27	74%	10.69	100%	3.81	91%	7.26	70%	3.02	74%	639	266
AVL global cache-obl.	5.17	29%	2.29	52%	5.30	49%	2.40	57%	4.76	46%	2.22	54%	419	196
AVL global	5.05	28%	2.27	51%	5.10	48%	2.31	55%	4.54	44%	2.13	52%	399	187
Plain cB ⁺	6.12	100%	2.71	100%	5.63	100%	2.39	100%	5.21	100%	2.20	100%	459	194
cB ⁺ global	5.08	83%	2.23	82%	4.15	74%	1.83	76%	3.77	72%	1.66	75%	332	146
Plain CSB ⁺	5.50	100%	2.14	100%	4.97	100%	1.82	100%	4.31	100%	1.61	100%	379	142
CSB ⁺ global	5.24	95%	2.12	99%	4.52	91%	1.70	94%	3.88	90%	1.50	93%	342	132

Names of trees are as in Table 2. The steady-state phase of the benchmark included 9.5×10^5 insertions, 6.6×10^4 deletions and 1.0×10^7 search operations (1.1×10^7 searches for the Searches only variation). The last two columns give the total time divided by the number of searches, directly comparable with the times in Table 2.

TABLE 4. Path lengths and memory use for the TPC-C benchmark reported in Table 3. The final tree has $n = 10\,228\,847 \approx 10^7$ keys (cf. Table 2), or $n = 9\,345\,023$ in the Searches only variation.

	Average path length in final tree									Size of final tree		
	Unmodified			Shuffled ins.			Searches only			Unmod	Shuffl	Srchs
	Nodes			Nodes			Nodes			MB	MB	MB
	Nodes	P'_1	P'_2	Nodes	P'_1	P'_2	Nodes	P'_1	P'_2	MB	MB	MB
Traditional red-black	22.8	22.3	17.3	24.0	23.9	19.9	22.7	22.0	18.0	156	156	143
Red-black local	22.8	13.6	11.5	24.0	14.0	13.9	22.7	13.5	12.3	187	182	164
Red-black global cache-obl.	22.8	12.8	7.1	24.0	15.5	9.9	22.7	11.8	6.1	176	202	204
Red-black global	22.8	11.4	4.4	24.0	13.8	6.9	22.7	10.5	3.3	171	161	163
Traditional AVL	22.7	22.1	17.6	22.8	22.8	17.6	22.6	21.9	17.7	156	156	143
AVL local	22.7	13.2	11.0	22.8	13.5	12.8	22.6	13.4	11.9	190	183	165
AVL global cache-obl.	22.7	13.2	7.4	22.8	12.5	7.0	22.6	11.8	6.1	193	202	204
AVL global	22.7	11.8	4.7	22.8	11.3	4.2	22.6	10.5	3.3	168	164	163
Plain cB ⁺	12.0	12.0	9.3	10.0	10.0	9.1	9.0	9.0	8.5	264	164	140
cB ⁺ global	12.0	12.0	3.8	10.0	10.0	5.4	9.0	9.0	3.0	264	179	179
Plain CSB ⁺	9.0	9.0	8.9	8.0	8.0	7.9	7.0	7.0	6.9	521	241	192
CSB ⁺ global	9.0	9.0	6.0	8.0	8.0	6.7	7.0	7.0	5.0	521	241	193

Finally, since large numbers of consecutive insertions commonly occur in practice, it is interesting to look at their effect by comparing the unmodified benchmark with the shuffled initial insertions. Table 4 shows one of the main effects: greatly increased space usage in the B-trees. This is unsurprising—most of the nodes are half full—although it may be interesting to note that CSB⁺-trees are even worse, consecutive insertions

more than doubling the space usage.⁸ Comparing running times (Table 3, columns ‘Unmodified’ and ‘Shuffled ins.’), the B-trees are somewhat slower after consecutive insertions,

⁸ Many of the node groups in the CSB⁺-trees are also half full after consecutive insertions. Rao and Ross [1] discuss other variants of CSB⁺-trees that have better space usage at the expense of performance.

but the binary search trees are actually slightly faster. Thus, optimized B-trees need special code for consecutive insertions, while binary search trees do not.

6. RELATED WORK

Most of the research on the effect of caching on 1D search trees has concentrated on variants of the B-tree. The simplest cache-sensitive B-tree variant is an ordinary B^+ -tree where the node size is chosen to match the size of a cache block (e.g. 64 or 128 bytes) [3]. Rao and Ross [1] have designed a more advanced version called the Cache-Sensitive B^+ -tree or CSB^+ -tree, which increases the fanout of the tree by removing most of the child pointers and storing the children of a node consecutively in memory. The CSB^+ -tree has been further optimized using a variety of techniques, such as prefetching [16], choosing the node size more carefully [2], storing only partial keys in nodes [17] and other key compression techniques [18]. (Many of these additional optimization techniques could also be applied to binary search trees and the relocation algorithms of the present article.)

More recently, Lee *et al.* [5,19] have made the T-tree cache-sensitive. The T-tree is an internal binary search tree where each node contains a up to a fixed number of keys that are consecutive, i.e. the tree has no other keys between the smallest and largest keys stored in a T-tree node. (In contrast, two keys that are consecutive in an internal B-tree node have a whole subtree of keys between them.) Unlike a traditional T-tree, the cache-sensitive T-tree (CST-tree) is not binary. Instead, at the outermost level the CST-tree is an m -way tree with cache-block-sized nodes (thus $m = O(B)$ where B is the cache block size). Each node stores a small implicit-pointer binary tree in an array, and its children are stored consecutively in memory as in the CSB^+ -tree. This structure is used as an index to ‘data nodes’, which (like traditional T-tree nodes) store a fixed number of consecutive keys in an array. The advantage of this separate index structure is that it only needs to store the maximum key of each data node, since only the maximum keys are required to search down the tree.

As in the CSB^+ -tree, a single structure modification (split in CSB^+ -trees, a special rotation in CST-trees) requires $O(B^2)$ time, where B is the cache block size, since it may need to modify all children of an m -way node (because they are stored consecutively). CST- and CSB^+ -trees have been compared experimentally by Kim *et al.* [20] and Lee *et al.* [5].

The above structures were optimized to only one level of the cache (i.e. one fixed cache block size—we call this a single-level model), often the one closest to the CPU. Rahman *et al.* [21] and Chen *et al.* [22] have examined B-trees in two-level memory hierarchies: both use one cache block size together with the page size of the computer (the page size is used for the TLB and for possibly storing pages on disk). The approach of Rahman *et al.* [21] and the ‘disk-first’ approach of Chen *et al.* [22] use page-sized B-tree nodes and alter the

internal structure of the node to be cache-optimized. In contrast, the ‘cache-first’ approach of Chen *et al.* [22] uses small-node B-trees (with cache-optimized node size), laying them out on disk pages intelligently, for instance keeping parents and children together. The on-disk layout of the cache-first approach very loosely resembles our local relocation, but it is based on splitting whole disk pages (recall that our approach only moves a constant number of nodes). Their approach does not easily generalize to binary search trees, since it is based on splitting pages of nodes after B-tree node splits and not rotations which would alter the depths of the nodes; it also requires additional parent and sibling links in the tree.

In the more generic multi-level model with more than two separate cache block sizes, research has concentrated on methods of laying out an existing tree statically [4,12].

Several B-trees have been proposed in the cache-oblivious model, including the original rather complex cache-oblivious B-tree by Bender *et al.* [23], its two simpler variants [24,25], and a string B-tree optimized for longer keys [26]. They are based on weight-balanced B-trees [27] and a dynamically updated sparse array called the packed-memory array [23,28], and their structure and rebalancing operations are dictated by the cache-oblivious memory layout. Some variants [25,29] use a static implicit-pointer binary search tree (a complete tree stored in a cache-oblivious layout without pointers) in addition to the packed-memory array. Rebalancing in all of these depends on rebuilding or copying parts of the structures (both the tree and the packed-memory array), and most of the complexity bounds are amortized. Unlike the cache-sensitive B-trees, the cache-oblivious trees are not direct extensions of the usual height-balanced B-trees [30] or (a, b) -trees [31]. Thus, much of the extensive literature on optimizing B-trees and, e.g. supporting concurrency and crash recovery in databases (Graefe [32] gives a recent survey), does not directly apply to these structures.

Bender *et al.* [33] have shown that the best cache-oblivious layout is never more than 44% worse in the number of block transfers than an optimal cache-sensitive layout, and the two converge when the number of levels of caches increases. However, the cache-sensitive model remains important, because the number of levels of caches with different block sizes is small in practice (at least currently—for instance, only two in the four computers we used for our experiments in this article). Several experiments where the two models are compared [21,24,34] support the intuition that (especially multi-level) cache-sensitive search structures are more efficient than cache-oblivious ones.

In a cache-sensitive binary search tree, the node size cannot be chosen as freely as in a B-tree. Instead, fixed-size nodes can be placed in memory so that each cache block contains nodes that are close to each other in the tree. Explicit-pointer binary search trees in the cache-oblivious model have been considered by Jiang *et al.* [35], who present a cache-oblivious splay tree based on periodically rearranging all nodes in memory.

For the cache-sensitive model, Oksanen and Malmi [11] give a single-level periodic-rearrangement algorithm for explicit-pointer binary trees. Gil and Itai [12] and Bender *et al.* [4] analyze a similar single-level layout, but extended to unbalanced trees and non-uniform search patterns. Gil and Itai [12] show that finding an optimal layout in this model when also optimizing for space is NP-complete, while Bender *et al.* [4] give an efficient layout algorithm that is almost optimal. Chilimbi *et al.* [36] study cache-sensitive data structures from a different point of view, giving semi-automatic tools for reorganizing tree-like structures in existing programs, using the same style of single-level layout as the others.

Recently, Askitis and Zobel [37] have examined the problem of storing variable-length strings in various search structures, including cache-sensitive binary trees. Unlike the structures mentioned above, their tree nodes are not fixed-size, since they contain the string keys (instead of pointers to them). One of the structures they use is a ‘clustered BST’, which is organized in the layout of Chilimbi *et al.* [36], but does not support dynamic cache-sensitive updates. They also propose an ‘array BST’, which stores the nodes in an array in order of insertion, which they argue is cache-efficient for a non-rebalanced binary search tree (though in our terminology, it is not cache-sensitive, since the cache block size is not used).

None of these binary-tree layouts consider the problem of maintaining a cache-sensitive layout in insertion or deletion, except via periodically rearranging the whole tree into the cache-sensitive layout.

7. BINARY TREES VERSUS B-TREES

To the best of our knowledge, the experiments presented above are the first ones to compare various kinds of cache-sensitive binary trees and B-trees with each other. Unfortunately, from the point of view of the main topic of this article, on average the cache-sensitive B-trees outperformed most of the binary search trees: often the best binary trees were about as fast as the cache-sensitive B-trees, and implementing multi-level rearrangement in the B-trees made them faster.

Because of the large number of pointers per stored element, the structure of binary search trees is not ideally suited to hierarchical memory. However, our goal in this research was to find out how much binary trees can be improved with regard to the cache while keeping their essential properties. We feel that this is important, because binary trees are very commonly used in practice.⁹

An algorithmic advantage of binary trees is that rotations are constant-time operations, unlike B-tree node splits or merges,

which, for a cache block size B_1 , take $O(B_1)$ time in B-trees with B_1 -sized nodes or $O(B_1^2)$ in the full CSB^+ -tree. The cache-sensitive T-tree (CST-tree) also uses rotations—although not ordinary rotations, since the outermost tree of a CST-tree is non-binary—but unlike our solution, it needs to spend $O(B_1^2)$ time on each level of the tree to re-establish its stricter memory layout. Thus, our binary search trees have good worst-case performance for a single insertion or deletion, which could be useful in a real-time application. Constant-time rotations may also improve concurrency: nodes are locked for a shorter duration. In fact, AVL trees were quite recently used as the basis of a fast and scalable concurrent search tree that applies optimistic concurrency control techniques [38].

CST-trees use small binary search trees as part of their structure, although not in a form that our cache-sensitive methods could directly improve. In principle our methods could be used to redesign the CST-tree, giving an alternative with constant-time rotations (instead of $O(B_1^2)$), but with less efficient searches, since we are not as aggressive as the CST-tree in avoiding pointers.

Many modern large-scale applications of search trees employ path copying: instead of modifying nodes in-place, updates are made by copying the root-to-leaf path. This path has minimal size in binary trees, as Soisalon-Soininen and Widmayer [6] showed in connection with a full-text index that uses path copying for efficient crash recovery and transaction rollback. A more recent example is the scalable and distributed flash-memory-based database Hyder [7], which also cites the small size of the copied path as the reason to use binary trees. Although more efficient in binary trees, path copying is also used with non-binary trees, for instance in the write-optimized B-tree [39]. Other well-known uses of path copying include persistent or multi-version search trees.

8. CONCLUSIONS

We have examined how to use knowledge of the hardware cache to improve the performance of binary search trees. To retain the essential properties of binary search trees, we set ourselves the following restrictions: structure modifications (rotations) must remain constant-time, and the rebalancing algorithm (which rotations to perform) or structure of the nodes must not be changed. This also makes our methods applicable not only to the usual AVL and red-black trees, but to any binary search tree that uses rotations for balancing.

The best improvement possible with our restrictions was approximated by the non-dynamic global relocation algorithm: 51–56% faster searches in large AVL and red-black trees in our experiments. Our dynamic solution was based on a memory-layout invariant dictating that cache blocks should contain at least α connected nodes. Its simplest form ($\alpha = 2$) gave 26–32% improvement, more than half of the much more complex non-dynamic approach; this was true for both random searches and the search pattern of the TPC-C benchmark. The

⁹ To mention only a few examples, pdfTeX uses AVL trees for font mapping, the OpenBSD kernel firewall uses red-black trees to store active network connections, many parts of the Linux kernel use red-black trees, and the ordered dictionary data structure in the Java and C++ standard libraries is usually implemented as a red-black tree.

improvement is also consistent with our theoretical analysis (Theorem 3.1), which predicted 33% improvement in path lengths with $\alpha = 2$, up to 50% if α is increased.

We showed that preserving the invariant increases the time complexity of insertion and deletion by only a constant. Our experiments showed that the improved search path length actually made deletions slightly faster, but insertions were at worst about twice as slow (this was heavily machine-dependent). We feel that in many applications this update cost is a reasonable price to pay for the increase in search efficiency.

We also combined our dynamic approach with a non-dynamic relocation algorithm, finding (experimentally) that the memory layout produced by the non-dynamic algorithm degraded relatively slowly in future insertions and deletions when our dynamic approach was in use. It thus seems viable to maintain an even better memory layout by executing the expensive non-dynamic algorithm periodically in addition to our dynamic approach, if the large worst-case cost is not a problem.

A natural extension of our invariant would be to handle multi-level caches in some way. However, combining the multi-level model with the restriction to constant-time structure modifications does not seem feasible: for instance, it would not be possible to move a cache-block-sized area of nodes to re-establish a higher-level invariant after a structure modification. A direction for future research could be to lift this constraint to create a dynamic multi-level cache-sensitive search tree (based on either a B-tree or a binary search tree) using the technique of preserving a memory-layout invariant while keeping the structure of the nodes and rebalancing strategy otherwise intact. (Recall that unlike our approach, CSB⁺-trees, CST-trees and cache-oblivious B-trees all require extensive changes to the underlying tree structure.)

Since path copying is most efficient in binary trees, another interesting topic would be to apply cache-sensitivity to a copy-on-write binary search tree. The solution of the present article is not directly suitable for this problem, since updates need to copy the entire root-to-leaf path, but perhaps the approach of using a memory-layout invariant would be useful, for instance to copy some extra nodes in the new path to preserve cache-sensitivity.

ACKNOWLEDGEMENTS

Preliminary versions of some of the results in this article were published in [13,14].

FUNDING

This research was partially supported by the Academy of Finland.

REFERENCES

- [1] Rao, J. and Ross, K.A. (2000) Making B⁺-Trees Cache Conscious in Main Memory. *2000 ACM SIGMOD Int. Conf. Management of Data*, Dallas, TX, USA, May 16–18, pp. 475–486. ACM Press, New York.
- [2] Hankins, R.A. and Patel, J.M. (2003) Effect of Node Size on the Performance of Cache-Conscious B⁺-Trees. *2003 ACM SIGMETRICS Int. Conf. Measurement and Modeling of Computer Systems*, San Diego, CA, USA, June 10–14, pp. 283–294. ACM Press, New York.
- [3] Rao, J. and Ross, K.A. (1999) Cache Conscious Indexing for Decision-Support in Main Memory. *25th Int. Conf. Very Large Data Bases (VLDB 1999)*, Edinburgh, Scotland, UK, September 7–10, pp. 78–89. Morgan Kaufmann, Burlington.
- [4] Bender, M.A., Demaine, E.D. and Farach-Colton, M. (2002) Efficient Tree Layout in a Multilevel Memory Hierarchy. *10th Annual European Symp. Algorithms (ESA 2002)*, Rome, Italy, September 17–21, Lecture Notes in Computer Science 2461, pp. 165–173. Springer, Berlin.
- [5] Lee, I., Lee, S. and Shim, J. (2011) Making T-trees cache conscious on commodity microprocessors. *J. Inf. Sci. Eng.*, **27**, 143–161.
- [6] Soisalon-Soininen, E. and Widmayer, P. (1999) Concurrency and Recovery in Full-Text Indexing. *String Processing and Information Retrieval Symposium (SPIRE 1999)*, Cancun, Mexico, September 21–24, pp. 192–198. IEEE Computer Society, Washington, DC.
- [7] Bernstein, P.A., Reid, C.W. and Das, S. (2011) Hyder – A Transactional Record Manager for Shared Flash. *5th Biennial Conf. Innovative Data Systems Research (CIDR 2011)*, Asilomar, CA, USA, January 9–12, pp. 9–20.
- [8] Haeupler, B., Sen, S. and Tarjan, R.E. (2009) Rank-Balanced Trees. *11th Algorithms and Data Structures Symposium (WADS 2009)*, Banff, Canada, August 21–23, Lecture Notes in Computer Science 5664, pp. 351–362. Springer, Berlin.
- [9] Soisalon-Soininen, E. and Widmayer, P. (2003) Single and Bulk Updates in Stratified Trees: An Amortized and Worst-Case Analysis. *Computer Science in Perspective: Essays Dedicated to Thomas Ottmann*, Lecture Notes in Computer Science 2598, pp. 278–292. Springer, Berlin.
- [10] Transaction Processing Performance Council (2010) TPC benchmark C. Standard Specification, Revision 5.11.
- [11] Oksanen, K. and Malmi, L. (1995) Memory reference locality and periodic relocation in main memory search trees. *5th Hellenic Conf. Informatics*. Greek Computer Society.
- [12] Gil, J. and Itai, A. (1999) How to pack trees. *J. Algorithms*, **32**, 108–132.
- [13] Saikkonen, R. and Soisalon-Soininen, E. (2008) Cache-sensitive memory layout for binary trees. *5th IFIP Int. Conf. Theoretical Computer Science (IFIP TCS 2008)*, Milan, Italy, September 8–10, IFIP International Federation for Information Processing 273, pp. 241–255. Springer, Berlin.
- [14] Saikkonen, R. (2009) Bulk Updates and Cache Sensitivity in Search Trees. PhD Thesis, Helsinki University of Technology.
- [15] Karlton, P. L., Fuller, S.H., Scroggs, R.E. and Kaehler, E.B. (1976) Performance of height-balanced trees. *Commun. ACM*, **19**, 23–28.
- [16] Chen, S., Gibbons, P.B. and Mowry, T.C. (2001) Improving Index Performance Through Prefetching. *2001 ACM SIGMOD Int. Conf. Management of Data*, Santa Barbara, CA, USA, May 21–24, pp. 235–246. ACM Press, New York.

- [17] Bohannon, P., McIlroy, P. and Rastogi, R. (2001) Main-memory Index Structures with Fixed-size Partial Keys. *2001 ACM SIGMOD Int. Conf. Management of Data*, Santa Barbara, CA, USA, May 21–24, pp. 163–174. ACM Press, New York.
- [18] Jin, R. and Chung, T.-S. (2010) Node Compression Techniques Based on Cache-Sensitive B⁺-Tree. *9th IEEE/ACIS Int. Conf. Computer and Information Science (ICIS 2010)*, Yamagata, Japan, August 18–20, pp. 133–138. IEEE Computer Society, Washington, DC.
- [19] Lee, I., Shim, J., Lee, S. and Chun, J. (2007) CST-trees: Cache sensitive T-trees. *12th Int. Conf. Database Systems for Advanced Applications (DASFAA 2007)*, Bangkok, Thailand, April 9–12, Lecture Notes in Computer Science 4443, pp. 398–409. Springer, Berlin.
- [20] Kim, K., Shim, J. and Lee, I. (2007) Cache Conscious Trees: How do they Perform on Contemporary Commodity Microprocessors. *5th Int. Conf. Computational Science and its Applications (ICCSA 2007)*, Kuala Lumpur, Malaysia, August 26–29, Lecture Notes in Computer Science 4705, pp. 189–200. Springer, Berlin.
- [21] Rahman, N., Cole, R. and Raman, R. (2001) Optimised Predecessor Data Structures for Internal Memory. *5th Workshop on Algorithm Engineering (WAE 2001)*, Aarhus, Denmark, August 28–31, pp. Lecture Notes in Computer Science 2141, pp. 67–78. Springer, Berlin.
- [22] Chen, S., Gibbons, P.B., Mowry, T.C. and Valentin, G. (2002) Fractal Prefetching B⁺-trees: Optimizing Both Cache and Disk Performance. *2002 ACM SIGMOD Int. Conf. Management of Data*, Madison, WI, USA, June 2–6, pp. 157–168. ACM Press, New York.
- [23] Bender, M.A., Demaine, E.D. and Farach-Colton, M. (2005) Cache-oblivious B-trees. *SIAM J. Comput.*, **35**, 341–358.
- [24] Brodal, G.S., Fagerberg, R. and Jacob, R. (2002) Cache Oblivious Search Trees via Binary Trees of Small Height. *13th Annual ACM-SIAM Symp. Discrete Algorithms (SODA 2002)*, San Francisco, CA, January 6–8, pp. 39–48. Society for Industrial and Applied Mathematics, Philadelphia.
- [25] Bender, M. A., Duan, Z., Iacono, J. and Wu, J. (2002) A Locality-Preserving Cache-Oblivious Dynamic Dictionary. *13th Annual ACM-SIAM Symp. Discrete Algorithms (SODA 2002)*, San Francisco, CA, USA, January 6–8, pp. 29–38. Society for Industrial and Applied Mathematics, Philadelphia.
- [26] Bender, M.A., Farach-Colton, M. and Kuszmaul, B. C. (2006) Cache-Oblivious String B-Trees. *25th ACM SIGACT-SIGMOD-SIGART Symp. Principles of Database Systems (PODS 2006)*, Chicago, IL, USA, June 26–28, pp. 233–242. ACM Press, New York.
- [27] Arge, L. and Vitter, J.S. (2003) Optimal external memory interval management. *SIAM J. Comput.*, **22**, 1488–1508.
- [28] Itai, A., Konheim, A.G. and Rodeh, M. (1981) A Sparse Table Implementation of Priority Queues. *8th Colloquium on Automata, Languages and Programming (ICALP 1981)*, Acre, Israel, July 13–17, Lecture Notes in Computer Science 115, pp. 417–431. Springer, Berlin.
- [29] Kasheff, Z. (2004) Cache-oblivious dynamic search trees. Master's Thesis, Massachusetts Institute of Technology.
- [30] Bayer, R. and McCreight, E. (1972) Organization and maintenance of large ordered indexes. *Acta Inform.*, **1**, 173–189.
- [31] Huddleston, S. and Mehlhorn, K. (1982) A new data structure for representing sorted lists. *Acta Inform.*, **17**, 157–184.
- [32] Graefe, G. (2010) Modern B-tree techniques. *Found. Trends Databases*, **3**, 203–402.
- [33] Bender, M.A., Brodal, G.S., Fagerberg, R., Ge, D., He, S., Hu, H., Iacono, J. and López-Ortiz, A. (2003) The Cost of Cache-Oblivious Searching. *44th Annual IEEE Symp. Foundations of Computer Science (FOCS 2003)*, Cambridge, MA, USA, October 11–14, pp. 271–282. IEEE Computer Society, Washington, DC.
- [34] He, B. and Luo, Q. (2008) Cache-oblivious databases: limitations and opportunities. *ACM Trans. Database Syst.*, **33**, doi:10.1145/1366102.1366105.
- [35] Jiang, W., Ding, C. and Cheng, R. (2004) Memory Access Analysis and Optimization Approaches on Splay Trees. *7th Workshop on Languages, Compilers and Run-time Support for Scalable Systems (LCR 2004)*, Houston, TX, USA, October 21–23, pp. 1–6. ACM Press, New York.
- [36] Chilimbi, T.M., Hill, M.D. and Larus, J.R. (1999) Cache-Conscious Structure Layout. *ACM SIGPLAN 1999 Conf. Programming Language Design and Implementation*, Atlanta, GA, USA, May 1–4, pp. 1–12. ACM Press, New York.
- [37] Askitis, N. and Zobel, J. (2011) Redesigning the string hash table, burst trie, and BST to exploit cache. *ACM J. Exp. Algorithms*, **15**, doi:10.1145/1671970.1921704.
- [38] Bronson, N.G., Casper, J., Chafi, H. and Olukotun, K. (2010) A Practical Concurrent Binary Search Tree. *15th ACM SIGPLAN Symp. Principles and Practice of Parallel Programming*, Bangalore, India, January 9–14, pp. 257–268. ACM Press, New York.
- [39] Graefe, G. (2004) Write-Optimized B-Trees. *30th Int. Conf. Very Large Data Bases (VLDB 2004)*, Toronto, Canada, August 31–September 3, pp. 672–683. Morgan Kaufmann, Burlington.